Gate 06 — IUCN Sync & Seed Data

Status: Not started **Preconditions:** Gate 03 complete (auth), Gate 04 complete (Admin usable for verification) **Unlocks:** Gate 07 (MVP Public Frontend) — frontend needs real data to demonstrate

---

Purpose

Two independent but related tasks that must both be complete before the MVP frontend is meaningful:

1. **IUCN Sync** — automated weekly pull of conservation assessments from the IUCN Red List API v4 into `ConservationAssessment` records. This is the only automated integration for MVP.
2. **Seed Data** — a management command that loads the baseline species dataset (~95–100 species) from a prepared CSV. This is a technical task, not a user story.

Both are in this gate because they share the Celery + Redis infrastructure and because the frontend demonstration at the ECA Workshop requires live IUCN data, not placeholder data.

Deliverables

- Celery worker and Celery Beat scheduler running in Docker Compose (the services exist from Gate 01 but Celery app configuration is completed here)
- `IUCNClient` class in `integration/clients/iucn.py` — wraps IUCN Red List API v4 with rate limiting and response caching (7-day Redis TTL)
- `iucn_sync` Celery task — fetches current assessment for each `Species` with `iucn_taxon_id` set; creates or updates `ConservationAssessment` with `source = 'iucn_official'`, `review_status = 'accepted'`; skips undescribed taxa without `iucn_taxon_id`; logs all errors to `SyncJob.error_log`
- `celery beat` schedule: `iucn_sync` runs weekly (Sunday 02:00 UTC)
- `SyncJob` lifecycle management: job creates a `SyncJob` record on start, updates it on completion or failure
- Management command `seed_species` — idempotently loads species from `data/seed/madagascar_freshwater_fish_seed.csv`
- Management command `load_reference_layers` — loads HydroSHEDS watershed and WDPA protected area shapefiles into PostGIS
- Management command `seed_localities` — loads species locality records from `data/seed/madagascar_freshwater_fish_localities_seed.csv` into `SpeciesLocality` model
- Management command `generate_map_layers` — serializes reference layer tables to static GeoJSON files for frontend map overlays

---

Technical Tasks (Non-User-Story)

Task 1: Celery Configuration

- Initialize Celery app in `config/celery.py`; import in `config/__init__.py`
- Configure broker (Redis) and result backend (Redis) via `django-environ`
- Configure `celery beat` with database scheduler (`django-celery-beat`) so schedules are manageable via Django Admin
- Write a smoke test: enqueue a no-op task and confirm the worker processes it

- Add `celery inspect ping` health check to `GET /api/v1/health/` response

Task 2: IUCN API Client

The IUCN Red List API v4 requires a token (free registration at iucnredlist.org).

`IUCNClient` interface:

```
class IUCNClient:
    def get_species_assessment(self, iucn_taxon_id: int) -> dict | None:
        """
        Returns the current assessment dict or None if not found.
        Raises IUCNAPIError on non-404 HTTP errors.
        Caches responses in Redis for 7 days.
        """

    def get_species_by_name(self, scientific_name: str) -> dict | None:
        """
        Used for name-based lookup during seed data import.
        Returns IUCN taxon record or None.
        """
```

- Rate limit: max 1 request/second (IUCN API requirement). Use `time.sleep(1)` between requests within a single task run; do not use per-request sleep for cached hits.
- Cache key format: `iucn:assessment:{iucn_taxon_id}` with 7-day TTL
- Store raw API response in cache; parse into `ConservationAssessment` fields in the task, not the client

Task 3: `iucn_sync` Celery Task

```
@app.task(bind=True, max_retries=3)
def iucn_sync(self):
    job = SyncJob.objects.create(job_type='iucn_sync', status='running',
started_at=now())
    try:
        species_qs = Species.objects.exclude(iucn_taxon_id__isnull=True)
        for species in species_qs.iterator():
            try:
                assessment_data =
client.get_species_assessment(species.iucn_taxon_id)
                if assessment_data:
                    _upsert_assessment(species, assessment_data, job)
                else:
                    job.records_skipped += 1
            except IUCNAPIError as e:
                job.error_log += f"Species {species.id}: {e}\n"
        job.status = 'completed'
    except Exception as e:
        job.status = 'failed'
        job.error_log += str(e)
        raise
    finally:
```

```
        job.completed_at = now()
        job.save()
```

`_upsert_assessment` creates or updates a `ConservationAssessment` with `source='iucn_official'`, using `update_or_create` keyed on `(species, source='iucn_official')`. Sets `review_status='accepted'` only if no existing record has `review_status='pending_review'` (do not overwrite a coordinator's pending review flag with an IUCN sync).

Task 4: Seed Data Management Command

Location: `species/management/commands/seed_species.py`

The seed CSV (`data/seed/madagascar_freshwater_fish_seed.csv`) is prepared separately from this gate (data preparation is the project lead's task). The management command is code; the CSV is data. See [Data Preparation Guide]() for column-by-column guidance.

CSV schema (21 columns):

Column	Maps to Field	Required	Validation
`scientific_name`	`scientific_name`	Yes	Unique key for upsert; non-empty
`authority`	`authority`	No	Null for undescribed taxa
`year_described`	`year_described`	No	Integer year; null for undescribed taxa
`family`	`family`	Yes	Non-empty string
`genus`	`genus`	Yes	Non-empty string
`endemic_status`	`endemic_status`	Yes	Enum: `endemic`/`native`/`introduced`
`iucn_status`	`iucn_status`	No	Enum: `EX`/`EW`/`CR`/`EN`/`VU`/`NT`/`LC`/`DD`/`NE`; null if k
`iucn_taxon_id`	`iucn_taxon_id`	No	Integer; enables IUCN sync for this spe
`population_trend`	`population_trend`	No	Enum: `increasing`/`stable`/`decreasing`/`unk`
`cares_status`	`cares_status`	No	Enum: `CCR`/`CEN`/`CVU`/`CLC`; null if not listed
`taxonomic_status`	`taxonomic_status`	Yes	Enum: `described`/`undescribed_morphospecies`/`sp`
`provisional_name`	`provisional_name`	No	Informal epithet for undescribed taxa (
`shoal_priority`	`shoal_priority`	No	`true`/`false`; default `false`
`fishbase_id`	`fishbase_id`	No	Integer; FishBase species ID
`synonyms`	*(not stored directly)*	No	Semicolon-separated list of junior sync names; informational for data curation
`distribution_narrative`	`distribution_narrative`	No	Free text description of range
`habitat_type`	`habitat_type`	No	Free text (e.g., `streams`, `rivers and lak`

`max_length_cm`	`max_length_cm`	No	Decimal; maximum total length in cent
`in_captivity`	`in_captivity`	No	`Y`/`N`; default `false`. Boolean flag for pub ExSituPopulation records exist
`captive_institutions`	*(not stored directly)*	No	Comma-separated institution names; i ExSituPopulation seeding
`notes`	*(not stored directly)*	No	Provenance notes for data curator; not

Columns NOT in CSV (set by the platform):

- `gbif_taxon_key` — populated by future GBIF integration, not at seed time
- `description`, `ecology_notes`, `morphology` — populated via IUCN sync or manual admin entry
- `taxon` FK — populated by a future taxonomy management command

Command behavior:

```
python manage.py seed_species --csv data/seed/madagascar_freshwater_fish_seed.csv [--dry-run]
```

- `--dry-run` validates the CSV and reports what would be created/updated without writing to the database
- Idempotent: keyed on `scientific_name`; updates existing records if a row already exists in the database
- Logs counts: N created, N updated, N skipped (validation errors), with error details per row
- Does not delete existing records not present in the CSV
- After loading species, attempts IUCN name lookup for any species without `iucn_taxon_id` set (uses `IUCNClient.get_species_by_name`); populates `iucn_taxon_id` where found

Task 5: `load_reference_layers` Management Command

Location: `species/management/commands/load_reference_layers.py`

```
python manage.py load_reference_layers \
    --watersheds data/reference/hydrobasins_madagascar_lev06.shp \
    --protected-areas data/reference/wdpa_madagascar.shp \
    [--simplify 0.001]
```

Behavior:

- Loads HydroBASINS shapefile into `Watershed` table, filtering to Madagascar features only
- Loads WDPA shapefile into `ProtectedArea` table, filtering to `ISO3 = 'MDG'`
- Applies `ST_Simplify` at specified tolerance (default 0.001 degrees, ~100m) to reduce vertex counts
- Idempotent: keyed on `hybas_id` (Watershed) and `wdpa_id` (ProtectedArea). Existing records are updated; new records are created; no records are deleted.
- Implements top-down watershed naming: match ~15-20 major basin names from published hydrological maps (Betsiboka, Tsiribihina, Mangoky, Onilahy, Mangoro, Sofia, etc.); auto-generate "Sub-basin of [parent name]" for unnamed features using Pfafstetter hierarchy. Must avoid "Sub-basin of Sub-basin of..." chains by resolving to the nearest named ancestor.
- `--dry-run` flag: reports what would be loaded without writing to database

- Logs: N watersheds created/updated, N protected areas created/updated, total vertex count before/after simplification

Data source files (must be prepared before running):

- `data/reference/hydrobasins_madagascar_lev06.shp` — extracted from HydroSHEDS Africa level 6, filtered to Madagascar bounding box
- `data/reference/wdpa_madagascar.shp` — extracted from WDPA monthly download, filtered to `ISO3 = 'MDG'`

Task 6: `seed_localities` Management Command

Location: `species/management/commands/seed_localities.py`

```
python manage.py seed_localities \
    --csv data/seed/madagascar_freshwater_fish_localities_seed.csv \
    [--dry-run]
```

CSV Schema (14 columns):

See [Data Preparation Guide]() for column-by-column guidance and example rows.

Column	Maps to Field	Required	Default	V
`scientific_name`	FK lookup → Species	Yes	—	Must match `Species.scientific` not found
`latitude`	`location` (y)	Yes	—	Decimal degrees WGS84; betw extent)
`longitude`	`location` (x)	Yes	—	Decimal degrees WGS84; betw extent)
`locality_name`	`locality_name`	Yes	—	Non-empty string (e.g., "Sakar
`locality_type`	`locality_type`	Yes	—	Enum: `type_locality/collectio`
`presence_status`	`presence_status`	No	`present`	Enum: `present/historically_present_e`
`water_body`	`water_body`	No	*(blank)*	Name of water body (e.g., "Sal
`water_body_type`	`water_body_type`	No	*(blank)*	Enum: `river/lake/stream/cave_s`
`coordinate_precision`	`coordinate_precision`	No	`exact`	Enum: `exact/approximate/locali`
`year_collected`	`year_collected`	No	*(null)*	Integer year of collection/obse
`collector`	`collector`	No	*(blank)*	Person or expedition name
`source_citation`	`source_citation`	Yes	—	Where the record comes from
`is_sensitive`	`is_sensitive`	No	`false`	`true/false`; set `true` for CR spec generalized for public display
`notes`	`notes`	No	*(blank)*	Free text

Columns NOT in CSV (auto-computed on import):

- `drainage_basin` — assigned via `ST_Contains` spatial query against loaded Watershed polygons
- `drainage_basin_name` — denormalized from FK on save
- `location_key` — deterministic coordinate key computed from lat/lng on save
- `location_generalized` — computed from `location` + `is_sensitive` on save (rounded to 0.1 degree)

Behavior:

- Idempotent: keyed on `(species__scientific_name, location, locality_type)`. Existing records are updated; new records are created; records not in the CSV are not deleted.
- `--dry-run` flag: validates CSV and reports what would be created/updated/skipped without writing to database
- Logs: N created, N updated, N skipped (validation errors), with error details per row
- Validation warning (not rejection): if `locality_type = "type_locality"` and the matched species has `taxonomic_status = "undescribed_morphospecies"`, log a warning
- Auto-assign `drainage_basin` FK: for each locality, query `Watershed.objects.filter(geometry__contains=point)`. If exactly one match, assign it. If zero matches, set to null and log a warning. If multiple matches, assign the smallest by `area_sq_km`.

Prerequisite: `load_reference_layers` must have been run before `seed_localities` for drainage basin assignment to work. If no Watershed records exist, `seed_localities` logs a warning and sets all `drainage_basin` FKs to null.

Companion document: The localities CSV must be accompanied by `docs/data-sources/locality-data-sourcing.md`, documenting provenance, geocoding methodology, precision levels, sensitivity decisions, and known gaps. See architecture proposal Section 8 for the recommended structure.

Task 7: `generate_map_layers` Management Command

Location: `species/management/commands/generate_map_layers.py`

```
python manage.py generate_map_layers --output-dir staticfiles/map-layers/
```

Behavior:

- Serializes all Watershed records to `staticfiles/map-layers/watersheds.geojson`
- Serializes all ProtectedArea records to `staticfiles/map-layers/protected-areas.geojson`
- GeoJSON includes all attribute fields needed for frontend display (name, designation, area, IUCN category, species count per watershed)
- Overwrites existing files (this is a regeneration command, not an append)
- Logs: file path, record count, file size for each generated layer

Full Seed Execution Order

The following sequence must be followed for a complete data load:

```
# 1. Load reference layers (watersheds + protected areas)
python manage.py load_reference_layers \
    --watersheds data/reference/hydrobasins_madagascar_lev06.shp \
    --protected-areas data/reference/wdpa_madagascar.shp
```

```
# 2. Load species (existing Gate 06 command)
python manage.py seed_species --csv data/seed/madagascar_freshwater_fish_seed.csv

# 3. Load species localities (requires both reference layers and species)
python manage.py seed_localities \
    --csv data/seed/madagascar_freshwater_fish_localities_seed.csv

# 4. Generate static GeoJSON files for frontend (requires reference layers loaded)
python manage.py generate_map_layers --output-dir staticfiles/map-layers/
```

User Stories

BE-06-1: IUCN Sync Runs and Produces Assessments

As a conservation coordinator, **I want** the platform to automatically keep IUCN assessments current, **so that** species profiles always reflect the latest Red List status without manual entry.

Acceptance Criteria:

Given a Species record for *Pachypanchax sakaramyi* with `iucn_taxon_id = 166478` **When** the `iucn_sync` task runs **Then** a `ConservationAssessment` record exists with `species = sakaramyi`, `source = 'iucn_official'`, `category` matching the current IUCN assessment, and `review_status = 'accepted'`

Given a Species record with `taxonomic_status = 'undescribed_morphospecies'` and `iucn_taxon_id = null` **When** the `iucn_sync` task runs **Then** no IUCN API call is made for that species; `job.records_skipped` increments by 1; no error is logged

Given the IUCN API returns a 500 error for one species during a sync run **When** the task processes that species **Then** the error is recorded in `SyncJob.error_log` with the species ID; the task continues processing remaining species; `job.status = 'completed'` (not `'failed'`) if at least one species succeeded

Given a species has an existing `ConservationAssessment` with `review_status = 'pending_review'` **When** the `iucn_sync` task runs and finds a new IUCN assessment for that species **Then** the existing assessment's `review_status` is not overwritten; a new `ConservationAssessment` record is created alongside the pending one; the coordinator's flag is preserved

Given the weekly sync job runs twice in a row without any IUCN data changes **When** the second run completes **Then** no duplicate `ConservationAssessment` records are created (upsert behavior)

BE-06-2: Sync Job Visible in Django Admin

As a Tier 5 administrator, **I want** to see sync job history in Django Admin, **so that** I can diagnose sync failures without server access.

Acceptance Criteria:

Given an `iucn_sync` task completed with 3 errors **When** a Tier 5 admin views the SyncJob list in Django Admin **Then** the job appears with `status = 'completed'`, `records_updated`, `records_skipped`, and `error_log` showing the 3 error messages

---

Out of Scope

- FishBase sync (post-MVP)
- CARES sync (post-MVP; CARES 2.0 not yet live)
- GBIF publishing (post-MVP)
- ZIMS CSV import pipeline (post-MVP)
- Automated retry for failed species within a sync run (retry on next weekly run is sufficient)

Gate Exit Criteria

Before marking Gate 06 complete:

1. `iucn_sync` task completes against the real IUCN API in a local dev environment (with a valid token in `.env`)
2. `seed_species` management command loads the species CSV without errors; `--dry-run` flag validates correctly
3. After seeding, `GET /api/v1/species/` returns ~95–100 species
4. After sync, at least the described species with `iucn_taxon_id` set have `ConservationAssessment` records
5. SyncJob records are visible in Django Admin
6. `load_reference_layers` loads HydroSHEDS and WDPA data correctly; ~60-100 watersheds and ~120-160 protected areas exist in the database; major basins are named
7. `seed_localities` loads the localities CSV correctly; drainage basin FK assignment works via spatial containment; `--dry-run` validates without writing
8. `generate_map_layers` produces valid GeoJSON files in `staticfiles/map-layers/`
9. Full seed execution order runs without error: `load_reference_layers` → `seed_species` → `seed_localities` → `generate_map_layers`
10. All acceptance criteria tests pass (mock the IUCN API in tests; do not call the live API in CI)
11. Invoke **@test-writer** to write tests covering: task retry on transient failure, idempotent upsert, skip behavior for undescribed taxa, CSV validation edge cases, coordinate validation (outside Madagascar bbox), idempotent re-import of localities, spatial FK assignment, sensitive coordinate generalization, type_locality + undescribed_morphospecies warning
12. Invoke **@security-reviewer** — IUCN API token must be stored in env vars, never committed; verify Redis caching does not leak data across requests
13. Invoke **@code-quality-reviewer** on task and client code